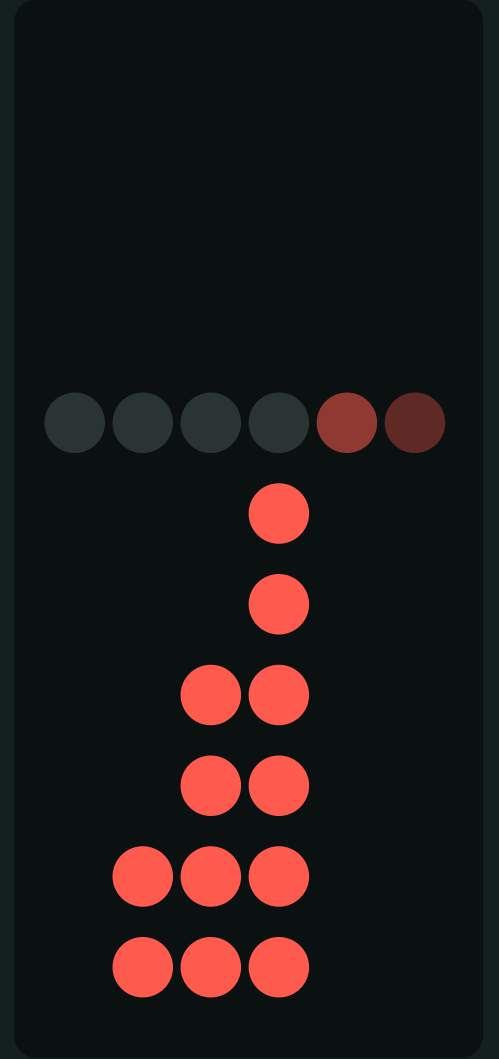


Stacker

Kendi arcade makineni yap: 64 LED'lik bir ekranı sıfırdan kablola, nasıl çalıştığını öğren, kodunu oku ve değiştir.



Bu derste ne yapacağız?

AVM'lerdeki "Stacker" ödül makinesini biliyor musunuz? Kayan bir bloğu doğru anda durdurup kuleyi tepeye kadar dikmeye çalışırsınız. Bugün aynı makinenin küçüğünü kendi ellerinizle yapacaksınız.

Bu dersin sonunda...

- Arduino'nun bir pininin ne yapabildiğini ve sınırlarını bileceksiniz.
- Bir LED'in neden dirence ihtiyaç duyduğunu hesaplayacaksınız.
- 16 kabloyla 64 LED'i nasıl kontrol ettiğimizi (tarama yöntemi) anlayacaksınız.
- Breadboard üzerinde 29 kablolu bir devreyi kurup test edeceksiniz.
- Bir oyunun kodunu okuyup kendi resminizi ve seviyenizi ekleyeceksiniz.

İçindekiler

- 01 Malzemeler ve her birinin görevi
- 02 LED, direnç ve Ohm yasası
- 03 8x8 matrisin içi
- 04 Tarama: 16 kabloyla 64 LED
- 05 Breadboard'u tanıyalım
- 06 Montaj: adım adım
- 07 Test ve hata ayıklama
- 08 Kodun yolculuğu
- 09 Oyunun kuralları ve seviyeler
- 10 Görevler ve sorular
- 11 Sözlük

Öğretmen için akış (35 dk): 0–3 kanca ve canlı demo · 3–6 breadboard'un sırrı · 6–20 montaj (rollerle) · 20–25 test · 25–32 oyun ve rekor turnuvası · 32–35 kodun kalbi (loop()).

Masadaki her parçanın bir görevi var

Devrede yedi çeşit parça kullanıyoruz. Önce her birinin ne işe yaradığını bilelim; montaj sırasında "bu kablo neden burada?" sorusunun cevabı hep bu sayfada.

PARÇA	ADET	GÖREVİ
Arduino UNO	1	Beyin. Kodu çalıştırır, pinlerinden 5 V (HIGH) ya da 0 V (LOW) verir, butonu okur, buzzer'a ses gönderir.
Breadboard	1	Lehim olmadan parçaları birbirine bağladığımız delikli tahta. İçindeki metal şeritler delikleri gruplar halinde birleştirir.
1088BS 8×8 LED matris	1	Ekranımız. 64 kırmızı LED, 16 bacak.
220 Ω direnç	8	LED'den geçen akımı sınırlar; hem LED'i hem Arduino'yu korur. Renk halkaları: kırmızı-kırmızı-kahverengi.
Tact buton (12×12)	1	Oyunun tek kontrolü: bloğu durdurur, oyunu başlatır.
Pasif buzzer	1	Ses efektleri ve melodiler. Altında yeşil devre kartı görünür.
Erkek-erkek jumper	29	Bağlantı kabloları. Sette 30 tane var, 1 yedek kalır.

Arduino UNO'nun pinleri

Dijital pinler (D0–D13): Kodla HIGH (5 V) ya da LOW (0 V) yapılır. D0 ve D1 bilgisayarla haberleşmeye ayrıldığı için kullanmıyoruz.

Analog pinler (A0–A5): Normalde gerilim ölçmek için, ama dijital pin gibi de kullanılabilirler. Biz öyle kullanıyoruz.

GND: Devrenin "eksi" ucu, 0 V. Akım her zaman GND'ye döner.

Pin bütçemiz

Kullanılabilir pin: 12 dijital + 6 analog = **18**

Matris: **16** · Buton: **1** · Buzzer: **1**

Toplam: **18 / 18**

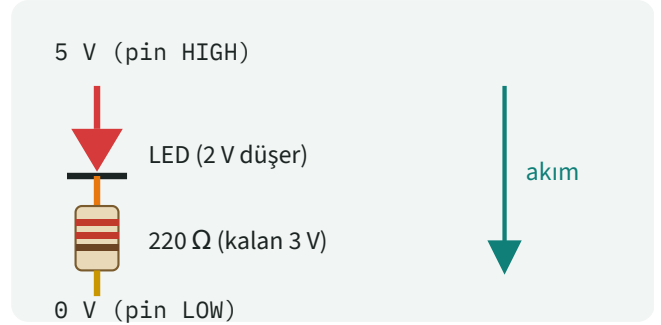
Her pin dolu! İleride ekran ya da sensör eklemek istersek "pin çoğaltma" yöntemlerine ihtiyacımız olacak.

Bir pinin sınırı: Arduino'nun bir pininden en fazla **40 mA** akım çekilebilir. Bunu aşarsak pin kalıcı olarak bozulabilir. Bütün devre tasarımı bu tek kurala göre şekilleniyor.

LED tek yönlü bir kapıdır

LED (Işık Yayan Diyot), akımın yalnızca **bir yönde** geçmesine izin verir: uzun bacak **anot (+)**, kısa bacak **katot (-)**. Akım anottan girip katottan çıkarsa ışık yanar; ters bağlarsak hiçbir şey olmaz (bozulmaz da).

Kırmızı bir LED üzerinde yaklaşık **2 V** "harcanır". Arduino 5 V verdiği için geriye 3 V kalır. Bu 3 V'u bir direnç karşılamazsa akım kontrolsüz yükselir: LED de pin de yanar.



Bir LED'in yanması için gereken yol: yüksek gerilimden düşük gerilime, LED'in doğru yönünden ve bir dirençten geçerek.

Ohm yasası ile hesaplayalım

Akım = Gerilim / Direnç

$$I = (5 \text{ V} - 2 \text{ V}) / 220 \, \Omega \approx 0,0136 \text{ A} \approx 13 \text{ mA} \quad // \text{ bir LED için}$$

Gerçekte Arduino pininin kendi iç direnci de biraz gerilim yer, bu yüzden bir LED'den yaklaşık **11–13 mA** geçer. Rahat bir parlaklık ve güvenli bir değer.

Soru: Bir satırda 8 LED birden yansaydı, o satırın pininden kaç mA çekilirdi? Pinin 40 mA sınırını aşar mıydı? (Cevap: $8 \times 12 \approx 96 \text{ mA}$. Sınırın iki katından fazla!)

Bu yüzden kodumuz bir satırda **aynı anda en fazla 3 LED** yakar: $3 \times 12 \approx 36 \text{ mA}$, sınırın altında.

Direnç renk kodu

220 Ω : kırmızı (2) · kırmızı (2) · kahverengi ($\times 10$) $\rightarrow 22 \times 10 = 220$. Sette 1 k Ω (kahverengi-siyah-kırmızı) ve 10 k Ω (kahverengi-siyah-turuncu) dirençler de var; karıştırmayın.

64 LED, ama sadece 16 bacak

64 LED'in her birini ayrı bağlasaydık 128 bacak gerekirdi. Matrisin içinde LED'ler ızgara gibi birbirine bağlanmış: aynı satırdaki 8 LED'in anotları birleşik, aynı sütundaki 8 LED'in katotları birleşik.



Kırmızı çizgiler satırlar (anot), turuncu çizgiler sütunlar (katot). Bir LED, satırı HIGH ve sütunu LOW olduğunda yanar.

Kural çok basit

Bir LED'i yakmak için o LED'in **satırını HIGH, sütununu LOW** yaparız. Diğer bütün satırlar LOW, diğer bütün sütunlar HIGH kalır; bu durumda hiçbir LED'den akım geçemez.

1088BS'de satırlar **ortak anot**. Koddaki SATIR_ANOT = true ayarı bunu söyler. (Kardeş model 1088AS'de tam tersi.)

Bacaklar neden karışık?

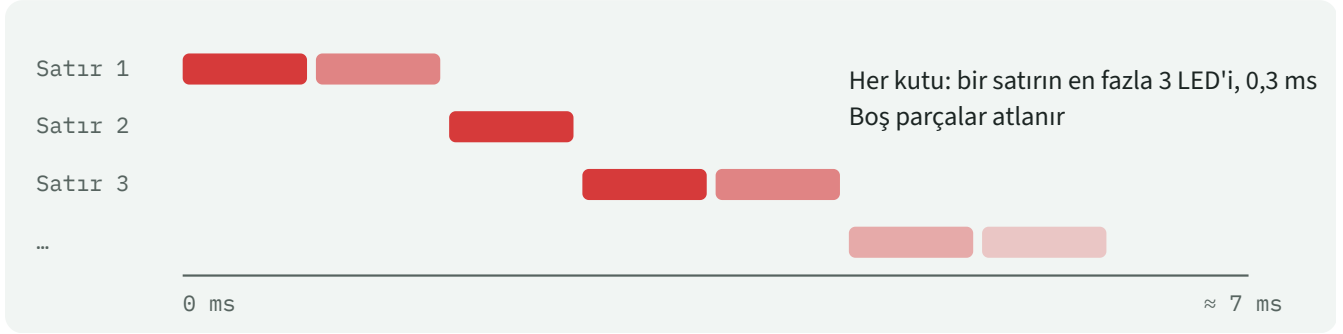
Matrisin içindeki çizgiler bacaklara düzensiz dağılmıştır: 1 numaralı bacak 5. satır, 13 numaralı bacak 1. sütun gibi. Biz kabloları breadboard'da **düz bir sırayla** takıyoruz; karışıklığı koddaki iki liste **çözüyor**:

```
int satirPinleri[8] = {A3, 12, 9, A0, 2, 8, 3, 6};
int sutunPinleri[8] = {13, 4, 5, A2, 7, A1, 11, 10};
```

"Kabloyu düzenli taktık, karmaşayı yazılım hallediyor."

Gözünüzü kandıran ekran

Bir resim çizmek için farklı satırlarda farklı LED'lerin yanması gerekir. Ama satırlar ve sütunlar paylaşıldığı için iki satırı aynı anda farklı desenle yakamayız. Çözüm: satırları çok hızlı sırayla yakmak.



Bir tam kare en fazla 8 satır $\times$ 3 parça = 24 adım $\times$ 0,3 ms $\approx$ 7 ms sürer. Saniyede 100'den fazla kare: göz titremeyi göremez.

Göz kalıcılığı

Göz, gördüğü ışığı saniyenin yaklaşık 1/25'i kadar "hatırlar". Bir LED saniyede 100 kez yanıp sönerse sürekli yanıyor sanırız. Televizyonlar, LED tabelalar ve sinema aynı hileyi kullanır.

Dene: Telefonunun kamerasıyla matrisi ağır çekimde çek. LED'lerin parça parça yandığını kendi gözünle göreceksin.

Neden 3'erli parçalar?

Satır pininden geçen akım, o anda yanan LED sayısı ile artar. 3 LED $\approx$ 36 mA, güvenli. Kalp ya da yüz gibi bir satırında 8 LED olan resimleri bile kod otomatik olarak 3'erli parçalara böler.

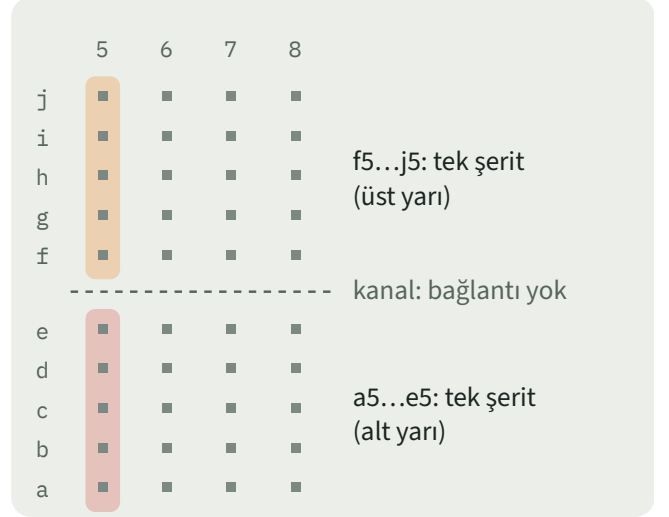
```
for (bas = 0; bas < 8; bas += 3) // 0-2, 3-5, 6-7
```

Görünmeyen metal şeritler

Breadboard'un sırrını bilmeden kablolama tutmaz. İki dakikanızı ayırın.

- **Sayılar (1–60)** sütunları, **harfler (a–j)** satırları gösterir. Her deliğin bir adresi var: b5, i17...
- **Aynı sayıdaki 5 delik birbirine bağlıdır:** a5-b5-c5-d5-e5 tek bir şerit, f5-g5-h5-i5-j5 başka bir şerit.
- Ortadaki **kanal** iki yarıyı ayırır: e5 ile f5 bağlı **değildir**.
- Kenarlardaki uzun **+ ve – hatları** boydan boya bağlıdır. Bizim devrede en üstteki – hattı GND.
- Bir şeride iki farklı hat takılırsa **kısa devre** olur.

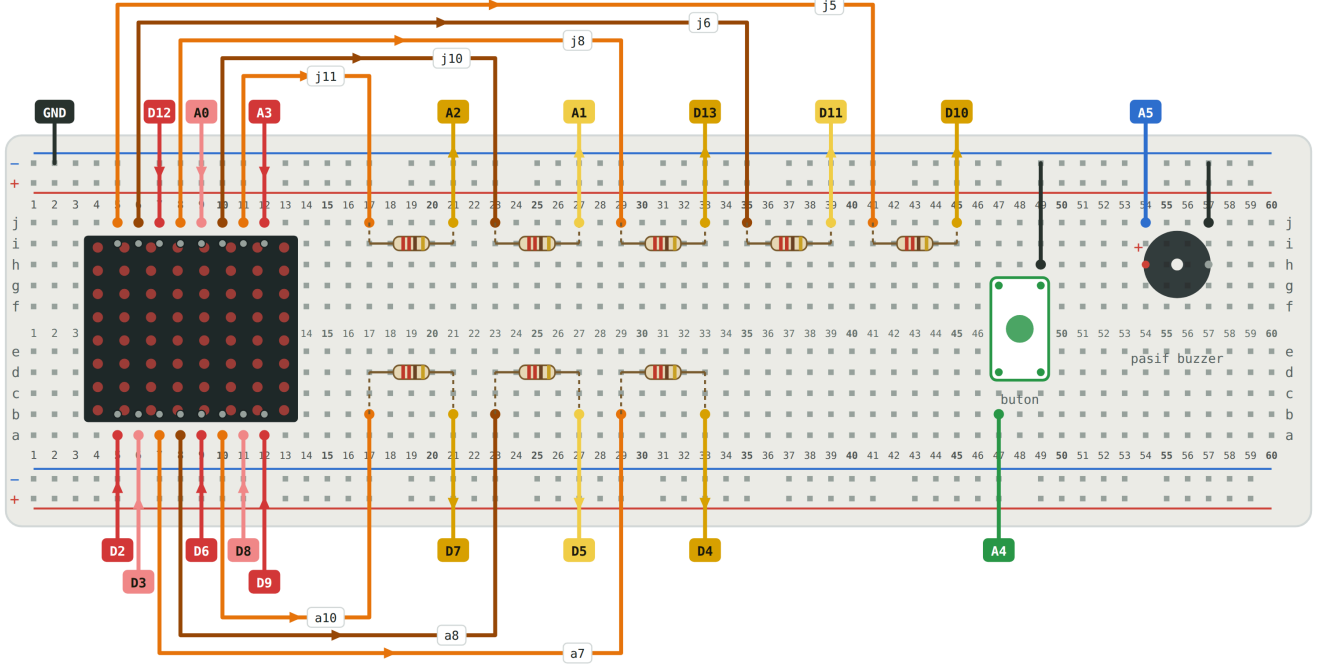
Dikkat: Bazı breadboard'larda + ve – hatları ortadan ikiye bölünmüştür. GND kablosunu, buton ve buzzer'ın bağlı olduğu yarıya takın.



Renkli bölgeler içinde birbirine bağlı delikler. 5. sütunun üst ve alt yarıları iki ayrı şerittir.

Tek kural: soldan sağa sırayla

Breadboard yatay, 1 numara solda; uzaktan yakına - + | j...f | e...a | - +. Matrisin bacakları b5-b12 ve i5-i12 deliklerine girer, "1088BS" yazısı size bakar. Her bacağı aynı sütundaki a ya da j deliğinden bağlarız.



Kablolar akımın yoluna göre renkli: **kırmızı** satır (Arduino → matris), **turuncu** sütun (matris → direnç), **sarı** dönüş (direnç → Arduino), **siyah** GND, **yeşil** buton, **mavi** buzzer.

ALT SIRA	ARDUİNO	YOL
a5	D2	düz kablo
a6	D3	düz kablo
a7	D4	b29 · d29-d33 · b33
a8	D5	b23 · d23-d27 · b27
a9	D6	düz kablo
a10	D7	b17 · d17-d21 · b21
a11	D8	düz kablo
a12	D9	düz kablo

ÜST SIRA	ARDUİNO	YOL
j5	D10	j41 · i41-i45 · j45
j6	D11	j35 · i35-i39 · j39
j7	D12	düz kablo
j8	D13	j29 · i29-i33 · j33
j9	A0	düz kablo
j10	A1	j23 · i23-i27 · j27
j11	A2	j17 · i17-i21 · j21
j12	A3	düz kablo

"Yol" sütunu dirençli hatlar için: *kablonun girdiği delik · direncin iki bacağı · Arduino kablosunun çıktığı delik.*

Buton, buzzer ve GND

GND	Arduino GND → en üstteki – hattı (buton ve buzzer'la aynı yarıya)
Buton	Bacaklar d47 d49 g47 g49 (kanalın üstünde) · b47 → A4 · h49 → –
Buzzer	Uzun bacak h54, kısa h57 · j54 → A5 · j57 → –

3 kişilik grup: herkes aynı anda çalışır

Rol 1 · Matrisçi

Matrisi takar, 8 kırmızı (düz)
kablo + GND.
9 kablo

Rol 2 · Üst dirençler

j5, j6, j8, j10, j11 hatları.
5 direnç · 10 kablo

Rol 3 · Alt dirençler

a7, a8, a10 + buton + buzzer.
3 direnç · 10 kablo

Önce test, sonra oyun

Mühendisler bir sistemi parça parça test eder. Oyunu yüklemekten önce matrisin her satırının ve sütununun çalıştığını kanıtlayacağız.

- 1 **Güç vermeden kontrol:** 16 delikte (a5–a12, j5–j12) birer bağlantı var mı? Dirençlerin iki bacağı farklı sütunda mı? GND takılı mı?
- 2 **matris_test.ino yükle:** Önce 8 yatay, sonra 8 dikey çizgi tek tek boydan boya yanmalı.
- 3 **Seri Monitör'ü aç (9600):** O an hangi hattın yandığını ve hangi delikte olduğunu yazar. Yanmayan çizgide yazan delik, kontrol edeceğin kablodur.
- 4 **Hepsi yanıyorsa stacker_ogrenci.ino yükle.** Açılışta ekran 3 kez yanıp söner ve 3 bip duyulur.

En sık karşılaşılan sorunlar

BELİRTİ	MUHEMEL SEBEP	ÇÖZÜM
Hiçbir LED yanmıyor	Kutup ayarı ya da matris ters takılı	Matrisin yazısı size bakıyor mu? Kodda SATIR_ANOT'u değiştir.
Bir çizgi hiç yanmıyor	Gevşek kablo ya da direnç	Seri Monitör'ün yazdığı deliğe bak, kabloyu bastır.
Görüntü karışık	Kablo yanlış deliğe ya da yanlış pine	Tabloyla tek tek karşılaştır.
Buzzer ve buton çalışmıyor	GND takılmamış ya da hat ortadan bölünmüş	GND'yi buton/buzzer'ın olduğu yarıya tak.
Buton çalışmıyor	A4 yuvası sıkı, kablo temas etmiyor	Kabloyu bastır ya da aynı pin olan SDA yuvasını kullan.
Melodi cızırtılı	Aktif buzzer takılmış	Altında yeşil kart görünen pasif buzzer'ı kullan.
Kule yandan büyüyor	Matrisin yönü farklı	Kodda donusDerecesi'ni 90, 180 ya da 270 yap.

Oyunun bütün kuralı 15 satır

Kod 10 bölüme ayrılmış. En önemlisi sonda: `loop()`. Oyun baştan sona bir hikâye gibi okunur.

```
int seviye = 1;
int can = 3;
while (can > 0 && seviye <= 9) {
    seviyeEkraniCiz(seviye);           // "L3"
    kalpAt(can);                       // kalp can kadar atar
    if (seviyeOyna(seviye)) {          // 8 kat tamamlandı mı?
        if (can < enFazlaCan) { canKazanildi(can, can + 1); can = can + 1; }
        seviye = seviye + 1;
    } else {                           // blok boşa düştü
        uzgunYuz();
        kalpKirildi(can, can - 1);
        can = can - 1;
    }
}
```

1 · Resimler kodun içinde çizili

```
const char* KALP[8] = {
    ".##.##.",
    "#####",
    "#####",
    "#####",
    ".#####.",
    "..#####.",
    "...##...",
    "....."
};
```

yanan LED, . sönük LED. Kodda gördüğünüz şekil, matriste aynen görünür.

Kodda ayrıca kırık kalp, mutlu, üzgün ve ağlayan yüz var. Kendi resminizi de aynı şekilde ekleyebilirsiniz.

Ekranın hafızası `bool ekran[8][8]`: 8×8'lik bir doğru/yanlış tablosu. `ekran[satır][sutun] = true` ise o LED yanar.

2 · Butonun "yeni" basışını yakalamak

```
bool simdi = digitalRead(butonPini);
bool basildi = (butonOnceki == HIGH
    && simdi == LOW
    && millis() - sonBasis > 50);
butonOnceki = simdi;
```

Buton `INPUT_PULLUP` modunda: Arduino pini içeriden 5 V'a çeker. Basılmadığında **HIGH**, basılınca pin GND'ye bağlanır ve **LOW** okunur. Bu yüzden butona ayrı direnç gerekmez.

"Önceki HIGH, şimdi LOW" = tam basıldığı an. 50 ms bekleme, butonun metal yayının titreşmesini (ark) filtreler.

3 · Bloğun düşüp düşmediği: aralık kesişimi

```
int sag      = sol + genislik - 1;
int yeniSol  = max(sol, altSol);
int yeniSag  = min(sag, altSag);
if (yeniSol > yeniSag) return false;
```

Her blok bir sol uç ve bir sağ uçtan ibaret. Üstteki ile alttaki bloğun ortak kısmı: solların büyüğünden sağların küçüğüne kadar. Sol, sağdan büyük çıkarsa hiç ortak kısım yok: blok boşa düştü.

4 · Ekranı döndürmek: modüler aritmetik

```
int derece = donusDerecesi % 360; // 450
→ 90
if (derece < 0) derece += 360; // -90
→ 270
donusAdimi = ((derece + 45) / 90) % 4; //
100 → 90
```

Matrisin etrafına dört kişi otursa, her biri kendi kenarını "yer" yapabilir. `donusDerecesi`'ne istediğiniz sayıyı yazın; kod onu 0, 90, 180 ya da 270'e indirir.

C++'ta negatif sayının modu negatif çıkar ($-90 \% 360 = -90$), bu yüzden araya +360 eklenir. +45 ise en yakın 90'a yuvarlama hilesidir.

5 · Hız ve seviye tablosu

Her seviyenin özellikleri dosyanın başındaki listelerde. Bir katta blok, `map()` ile ilk kattaki hızdan son kattaki hıza doğru yavaş yavaş hızlanır:

```
int hiz = map(kat, 0, 7, seviyeHizBas[i], seviyeHizSon[i]);
```

6 · Kalıcı rekor: EEPROM

Arduino kapatılınca değişkenler silinir. Rekoru korumak için Arduino'nun içindeki küçük kalıcı hafızaya, **EEPROM**'a yazıyoruz. Elektrik kesilse bile rekor durur.

```
if (ulasilan > rekor) EEPROM.write(0, ulasilan);
```

7 · Ses: frekans = nota

`tone(buzzerPini, 440, 200)` buzzer'ı saniyede 440 kez titreştirir: bu **La** notasıdır, orkestraların akort sesi. Frekansı iki katına çıkarırsanız aynı nota bir oktav inceler.

Kodun bölümleri

BÖLÜM	NE YAPAR
1 Pinler	Hangi parçanın hangi pine bağlı olduğu
2 Ayarlar	Kutup, dönüş derecesi, ayna, can sayıları
3 Seviyeler	9 seviyenin hızı, genişliği, rüzgâr ve hayalet özellikleri
4 Resimler	Kalpler, yüzler, rakamlar
5 Ekran hafızası	8×8 tablo ve çizim fonksiyonları
6 Matrisi yakmak	Tarama, döndürme, 3'erli parçalar
7 Buton ve bekleme	Basış algılama, ekranı göstererek bekleme
8 Animasyonlar	Kalp atışı, üzgün yüz, kırık kalp, flaş
9 Seviyeyi oynat	Kayan blok, kesme, düşme kontrolü
10 Ana program	<code>setup()</code> ve <code>loop()</code>

9 seviye, 5 kalp

- Butona bas: geri sayım, sonra oyun başlar. **3 canla** başlarsın.
- Kayan bloğu durdur. Alttaki bloğun üstüne gelmeyen kısım kesilir, blok daralır.
- **8 katı** tamamla: seviye atlarsın, **+1 can** (en fazla 5).
- Blok boşa düşerse: mutlu yüz üzülür, kalp kırılır, **1 can** gider ve seviye baştan başlar.
- Can biterse oyun biter; ulaştığın seviye rekorsa kalıcı hafızaya yazılır.
- 9 seviyeyi bitiren **şampiyon** olur.

Ekranlar: L3 = 3. seviye · ♥3 = 3 can.

SEVİYE	BLOK	ÖZEL
1-2	3	Isınma, sadece hız artar
3	3	Rüzgâr: blok aniden yön değiştirir
4	3	Hayalet: blok arada görünmez olur
5	2	Dar blok
6	3	Rüzgâr, hızlı
7	2	Hayalet + dar
8	2	Rüzgâr + hayalet
9	2	Final: en hızlı, rüzgâr + hayalet

Gerçek hayattan: AVM'deki ödül makineleri çoğu zaman tamamen dürüst değildir: son katlarda blok hızlanır ya da kazanma oranı operatör tarafından ayarlanır. Kendi makinenizi yaptıktan sonra şunu düşünün: bir oyunun "adil" olduğunu nasıl anlarız? Kodu okuyabilen kişi bunu bilir.

Şimdi sıra sizde

9. sınıf

Görev 1 · Kendi resmin: Kareli kâğıda 8×8 bir kare çiz ve kendi resmini tasarla (yıldız, ok, baş harfin...). Kodda MUTLU dizisinin yerine # ve . ile yaz. Kaybedince senin resmin görünecek.

Soru 1: Bir satırda aynı anda 3 LED yanarken pinden yaklaşık kaç mA geçer? Neden 4 değil de 3?

Soru 2: Breadboard'da a17 ile e17 bağlı mıdır? Peki e17 ile f17?

10. sınıf

Görev 2 · Kendi seviyen: Seviye listelerinden birini değiştir: hız, genişlik, rüzgâr, hayalet. Başka bir grup senin seviyeni oynasın. En zor ama hâlâ geçilebilir seviyeyi kim tasarlayacak?

Soru 3: `donusDerecesi = -630` yazarsak kule hangi kenardan büyür? Önce kâğıtta hesapla, sonra dene. (İpucu: $-630 \% 360 = -270$.)

Soru 4: `butonaBasildi()` fonksiyonundaki "50 ms" kontrolünü silersek ne olur? Neden?

Soru 5 · Tartışma: Son katta bloğu gizlice bir adım kaydıran bir "AVM modu" yazmak kaç satır sürer? Böyle bir makine dürüst müdür?

Gelecek hafta

Bugün pinlerin hepsini kullandık. Arduino'nun pini bitince ne yapılır? Bunun cevabı "pin çoğaltma": örneğin bir 74HC595 kaydırma yazmacıyla 3 pinden 8 çıkış elde edilebilir. Makinemize yeni özellikler eklemenin yolu buradan geçiyor.

Bilmen gereken kelimeler

Anot / Katot	LED'in artı (uzun bacak) ve eksi (kısa bacak) ucu. Akım anottan katoda akar.
HIGH / LOW	Pinin 5 V ya da 0 V olması; kodda <code>digitalWrite(pin, HIGH)</code> .
GND	Toprak, 0 V. Devrenin dönüş yolu.
mA (miliamper)	Akımın birimi. $1000 \text{ mA} = 1 \text{ A}$. Arduino pini en fazla 40 mA.
Ohm yasası	Akım = Gerilim / Direnç. Direnç seçmenin temel formülü.
Ortak anot	Bir satırdaki LED'lerin anotlarının tek bacakta birleşmesi (1088BS).
Tarama (multiplexing)	Ekranı parça parça, çok hızlı sırayla yakarak bütün resmi göstermek.
Göz kalıcılığı	Gözün ışığı kısa bir süre "hatırlaması". Taramanın çalışma sebebi.
INPUT_PULLUP	Pinin içeriden 5 V'a çekilmesi; buton basılınca LOW okunur.
Ark (bounce)	Butona basıldığında metal yayın birkaç milisaniye titreşmesi.
EEPROM	Elektrik kesilince silinmeyen küçük kalıcı hafıza.
Frekans	Saniyedeki titreşim sayısı (Hz). Sesin inceliğini belirler.
Modüler aritmetik (%)	Bölmeden kalan. $450 \% 360 = 90$.
Kısa devre	Akımın bir yükten geçmeden doğrudan dönmesi. Parçalara zarar verir.

Dosyalar: `matris_test.ino` (bağlantı testi) · `stacker_ogrenci.ino` (derste kullanılan sürüm) · `stacker_pro.ino` (aynı oyunun bit maskeleriyle yazılmış ileri sürümü).